

Action Desk ADCD Dialog v1

Version v1 - [GitHub dark PDF edition](#)

ADCD means **Action Desk Communication Daemon**. It exposes a small local HTTP API that allows scripts, background jobs, and local tools to show native Action Desk dialogs.

Endpoint

TEXT
POST http://127.0.0.1:4788/adcd/v1/dialog

Default host and port:

Setting	Default	Notes
Host	127.0.0.1	Local machine only
Network host	0.0.0.0	Used when network access is enabled
Port	4788	Configurable, clamped between 1024 and 65535

Request body must be JSON.

Response:

JSON
<pre>{ "button": "ok", "values": {} }</pre>

For invalid requests:

JSON
<pre>{ "ok": false, "error": "Invalid JSON: ..." }</pre>

Top-level JSON shape

JSON
<pre>{ "title": "Action Desk", "icon": "info", "message": "Hello from ADCD.", "sound": "info", "fields": [], "buttons": {}, "timeout_ms": 5000, "dismissible": true, "await_result": true }</pre>

Top-level properties

Property	Type	Default	Notes
<code>title</code>	string	<code>Action Desk</code>	Dialog title
<code>icon</code>	string	<code>null</code>	Icon name, resource path, app icon path, or absolute file path
<code>message</code>	string	<code>null</code>	Main message text
<code>sound</code>	string/object/bool	<code>null</code>	Plays a WAV sound once when dialog is shown
<code>fields</code>	array	<code>[]</code>	Form fields or row objects
<code>buttons</code>	object	default buttons	Button model/configuration
<code>width</code>	number/string	<code>auto</code>	Supports pixels, <code>auto</code> , <code>vw</code> , e.g. <code>520</code> , <code>"70vw"</code>
<code>height</code>	number/string	<code>auto</code>	Supports pixels, <code>auto</code> , <code>vh</code> , e.g. <code>"60vh"</code>
<code>min_width</code>	number	<code>240</code>	Minimum dialog width
<code>max_width</code>	number	<code>900</code>	Maximum dialog width
<code>min_height</code>	number/string	<code>null</code>	Optional minimum height
<code>max_height</code>	number/string	<code>80vh</code>	Maximum dialog height
<code>padding</code>	number/string/array	<code>0</code>	Body padding; CSS-like 1-4 values
<code>layout</code>	object	column layout	Top-level layout config
<code>position</code>	string	<code>center</code>	Passed to Action Desk dialog positioner
<code>dismissible</code>	bool	<code>true</code>	Allows close button, Alt+F4, and Esc to dismiss
<code>always_on_top</code>	bool	<code>false</code>	Keeps dialog above other windows
<code>show_in_taskbar</code>	bool	<code>true</code>	Uses a taskbar-visible frame when true
<code>await_result</code>	bool	<code>true</code>	If false, HTTP returns after the dialog is shown
<code>timeout_ms</code>	number	see timeout rules	Auto-closes with button <code>timeout</code>

Sound

`sound` is optional. Missing or `null` means no sound.

Simple form:

```
JSON
{
  "sound": "success"
}
```

Object form:

```
JSON
{
  "sound": {
    "src": "warning"
  }
}
```

Boolean form:

```
JSON
{
  "sound": true
}
```

`true` maps to `default`. `false` disables sound.

Recommended sound values

These are the recommended built-in/conventional names:

Value	Expected file
success	sound/success.wav
warning	sound/warning.wav
info	sound/info.wav
error	sound/error.wav
default	sound/default.wav, falling back to sound/info.wav

Disable values:

TEXT
none, off, no, false, 0, null, ""

Example:

JSON
<pre>{ "sound": "none" }</pre>

Sound file resolution

ADCD supports WAV only for v1.

For "sound": "warning", ADCD tries:

TEXT
<APP_FOLDER>/sound/warning.wav sound/warning.wav from bundled resources

For an absolute path, ADCD uses the file directly:

JSON
<pre>{ "sound": "C:/Users/ahad/Desktop/warning.wav" }</pre>

Use short PCM WAV files. MP3 is not supported in v1.

Timeout behaviour

Message-only dialogs:

timeout_ms	Behaviour
missing/null	Auto-close after 5000 ms
0	No auto-close
positive number	Auto-close after that many ms

Dialogs with fields/forms:

timeout_ms	Behaviour
missing/null	No auto-close
0	No auto-close
positive number	Auto-close after that many ms

When timeout happens:

JSON
<pre>{ "button": "timeout", "values": {} }</pre>

await_result

When `await_result` is `true`, ADCD waits until the user clicks a button, dismisses, or the dialog times out.

When `await_result` is `false`, ADCD returns immediately after showing the dialog:

JSON

```
{
  "button": "shown",
  "values": {}
}
```

This is useful for notification-style dialogs.

Layout

Top-level layout:

JSON

```
{
  "layout": {
    "direction": "column",
    "gap": 8,
    "padding": 12,
    "align_items": "start"
  }
}
```

Supported layout properties:

Property	Values	Notes
<code>direction</code>	<code>column</code> , <code>row</code>	<code>column</code> is the practical default
<code>gap</code>	number	Space between children
<code>padding</code>	number/string/array	Internal padding
<code>align_items</code>	<code>start</code> , <code>center</code>	Useful mainly for row layout

Padding/margin values support CSS-like shorthand:

JSON

```
{ "padding": 12 }
```

JSON

```
{ "padding": "8 12" }
```

JSON

```
{ "padding": [8, 12, 8, 12] }
```

Fields

`fields` is a vertical stack. Each item can be a direct field or a row object.

Direct field:

JSON

```
{
  "type": "input",
  "name": "username",
  "label": "Username"
}
```

Row object:

```
JSON

{
  "layout": {
    "direction": "row",
    "gap": 12,
    "align_items": "center"
  },
  "items": [
    { "type": "input", "name": "first_name", "label": "First name" },
    { "type": "input", "name": "last_name", "label": "Last name" }
  ]
}
```

Field-level `margin` controls outside spacing around that field/row.

Field types

Type	Purpose	Returns value?
<code>input</code>	Single-line text input	yes
<code>password</code>	Password input	yes
<code>number</code>	Text field with numeric validation by default	yes
<code>textarea</code>	Multi-line text input	yes
<code>select</code>	Dropdown select	yes
<code>checkbox</code>	Boolean checkbox	yes
<code>radio</code>	Radio option group	yes
<code>label</code>	Static display label	no
<code>text</code>	Static display text	no
<code>divider</code>	Horizontal divider	no
<code>image</code>	Static image/icon	no

Common field properties

Property	Type	Notes
<code>type</code>	string	Field/component type
<code>name</code>	string	Required if the value should be returned
<code>label</code>	string	Field label
<code>label_position</code>	string	<code>top</code> , <code>left</code> , <code>right</code> , <code>placeholder</code> , <code>none</code>
<code>value</code>	string	Initial value; for <code>label</code> / <code>text</code> , this is display text
<code>placeholder</code>	string	Placeholder text for input fields
<code>checked</code>	bool	Initial value for checkbox
<code>options</code>	array	Used by select/radio
<code>width</code>	number	Fixed component width where supported
<code>height</code>	number	Fixed component height where supported
<code>src</code>	string	Image source for <code>image</code> field
<code>margin</code>	number/string/array	Outside spacing

Validation properties

Property	Type	Notes
<code>required</code>	bool	Required field validation
<code>data_type</code>	string	<code>string</code> , <code>int</code> , <code>float</code> , <code>bool</code>
<code>min_len</code>	number	Minimum string length
<code>max_len</code>	number	Maximum string length
<code>min</code>	number	Minimum numeric value
<code>max</code>	number	Maximum numeric value
<code>allowed_values</code>	array	Allowed text values
<code>case_sensitive</code>	bool	Used with <code>allowed_values</code>

For `type: "number"`, default effective data type is `float`.

Options

String options:

JSON
<pre>{ "options": ["low", "medium", "high"] }</pre>

Object options:

JSON
<pre>{ "options": [{ "value": "low", "label": "Low" }, { "value": "high", "label": "High" }] }</pre>

Buttons

Default behaviour:

Dialog type	Default button model
Message-only	<code>acknowledge</code>
Dialog with fields	<code>confirm</code>

Button models:

Model	Behaviour
<code>acknowledge</code>	One neutral button
<code>confirm</code>	Cancel + submit button
<code>custom</code>	Uses only <code>buttons.items</code>

Button config:

JSON

```
{
  "buttons": {
    "model": "confirm",
    "layout": "right",
    "gap": 8,
    "show border": true,
    "confirm": {
      "id": "save",
      "label": "Save",
      "background": "#2563eb",
      "color": "#ffffff"
    },
    "cancel": {
      "id": "cancel",
      "label": "Cancel"
    }
  }
}
```

Button properties:

Property	Type	Notes
id	string	Returned as button
label	string	Button text
role	string	submit, cancel, neutral
default	bool	Sets default/Enter button
color	string	Text color
background	string	Background color
padding	number/string/array	Button padding
font_size	number	Button font size
border_radius	number	FlatLaf arc radius

Button layout values:

TEXT

left, center, right, fill

Image field

JSON

```
{
  "type": "image",
  "src": "warning",
  "width": 64,
  "height": 64
}
```

Image source can be:

TEXT

absolute file path
app icons folder file/name
bundled icon resource/name
plain icon name such as warning, success, info, error

Supported image formats:

TEXT

svg, png, jpg, jpeg

If one of width or height is missing, the other value is reused. If both are missing, the default is 64x64.

Result values

Submit button on a form returns named field values:

JSON

```
{
  "button": "ok",
  "values": {
    "username": "ahad",
    "remember": true
  }
}
```

Cancel/neutral buttons return empty values:

JSON

```
{
  "button": "cancel",
  "values": {}
}
```

Dismiss returns:

JSON

```
{
  "button": "dismiss",
  "values": {}
}
```

Examples

1. Simple message

JSON

```
{
  "title": "Done",
  "icon": "success",
  "message": "Backup completed successfully.",
  "timeout_ms": 3000,
  "dismissible": true
}
```

2. Message with sound

JSON

```
{
  "title": "Warning",
  "icon": "warning",
  "sound": "warning",
  "message": "Disk space is getting low.",
  "buttons": {
    "model": "acknowledge",
    "acknowledge": {
      "label": "OK"
    }
  }
}
```

3. Fire-and-forget notification

JSON

```
{
  "title": "Task Started",
  "icon": "info",
  "sound": "info",
  "message": "The sync task has started in the background.",
  "await_result": false,
  "timeout_ms": 2500,
  "show_in_taskbar": false
}
```


Immediate response:

JSON

```
{
  "button": "shown",
  "values": {}
}
```

4. Confirm dialog

JSON

```
{
  "title": "Delete file?",
  "icon": "warning",
  "sound": "warning",
  "message": "This file will be moved to trash.",
  "buttons": {
    "model": "confirm",
    "confirm": {
      "id": "delete",
      "label": "Delete",
      "background": "#dc2626",
      "color": "#ffffff"
    },
    "cancel": {
      "id": "cancel",
      "label": "Cancel"
    }
  }
}
```

5. Simple input form

JSON

```
{
  "title": "Create user",
  "icon": "user",
  "sound": "info",
  "message": "Enter the new user details.",
  "width": 420,
  "padding": 16,
  "layout": {
    "gap": 10
  },
  "fields": [
    {
      "type": "input",
      "name": "username",
      "label": "Username",
      "required": true,
      "min_len": 3,
      "max_len": 30
    },
    {
      "type": "password",
      "name": "password",
      "label": "Password",
      "required": true,
      "min_len": 8
    },
    {
      "type": "checkbox",
      "name": "enabled",
      "label": "Enable user immediately",
      "checked": true
    }
  ],
  "buttons": {
    "model": "confirm",
    "confirm": {
      "id": "create",
      "label": "Create"
    }
  }
}
```

6. Row layout with validation

JSON

```
{
  "title": "Server limit",
  "icon": "server",
  "sound": "info",
  "message": "Set the worker and retry limits.",
  "width": 520,
  "padding": "16 18",
  "layout": {
    "gap": 12
  },
  "fields": [
    {
      "layout": {
        "direction": "row",
        "gap": 12,
        "align_items": "center"
      },
      "items": [
        {
          "type": "number",
          "name": "workers",
          "label": "Workers",
          "required": true,
          "data_type": "int",
          "min": 1,
          "max": 20
        },
        {
          "type": "number",
          "name": "retries",
          "label": "Retries",
          "required": true,
          "data_type": "int",
          "min": 0,
          "max": 10
        }
      ]
    },
    {
      "type": "select",
      "name": "priority",
      "label": "Priority",
      "required": true,
      "options": [
        { "value": "low", "label": "Low" },
        { "value": "normal", "label": "Normal" },
        { "value": "high", "label": "High" }
      ],
      "value": "normal"
    }
  ],
  "buttons": {
    "model": "confirm",
    "confirm": {
      "id": "apply",
      "label": "Apply"
    },
    "cancel": {
      "id": "cancel",
      "label": "Cancel"
    }
  }
}
```

7. Advanced mixed content dialog

JSON

```
{
  "title": "Deployment checklist",
  "icon": "build",
  "sound": {
    "src": "success"
  },
  "width": 620,
  "max_height": "80vh",
  "padding": 18,
  "layout": {
    "gap": 12
  },
  "message": "Review the deployment details before continuing.",
  "fields": [
    {
      "layout": {
        "direction": "row",
        "gap": 12,
        "align_items": "center"
      },
      "items": [
        {
          "type": "image",
          "src": "success",
          "width": 48,
          "height": 48
        },
        {
          "type": "text",
          "value": "Build completed. Confirm the environment and release notes."
        }
      ]
    },
    {
      "type": "divider"
    },
    {
      "type": "select",
      "name": "environment",
      "label": "Environment",
      "required": true,
      "options": [
        { "value": "staging", "label": "Staging" },
        { "value": "production", "label": "Production" }
      ],
      "value": "staging"
    },
    {
      "type": "textarea",
      "name": "notes",
      "label": "Release notes",
      "required": true,
      "min_len": 10,
      "height": 120
    },
    {
      "type": "checkbox",
      "name": "confirmed",
      "label": "I have checked the deployment notes",
      "required": true
    }
  ],
  "buttons": {
    "model": "confirm",
    "layout": "right",
    "confirm": {
      "id": "deploy",
      "label": "Deploy",
      "background": "#2563eb",
      "color": "#ffffff",
      "padding": "6 14",
      "border_radius": 8
    },
    "cancel": {
      "id": "cancel",
      "label": "Cancel"
    }
  }
}
```

PowerShell example

POWERSHELL

```
$body = @'
{
  "title": "Backup Complete",
  "icon": "success",
  "sound": "success",
  "message": "Backup completed successfully.",
  "timeout_ms": 3000
}
'@

Invoke-RestMethod `
-Method Post `
-Uri "http://127.0.0.1:4788/adcd/v1/dialog" `
-ContentType "application/json" `
-Body $body
```

v1 notes

- ADCD Dialog v1 is intentionally practical and form-focused.
- Generic top-level `children` is not part of v1.
- `fields` supports direct fields and row objects.
- `label` and `text` display content using `value`.
- `justify_content` is not part of v1 layout behaviour.
- Sound is WAV-only in v1.